

Healthcare AI Platform

Project Progress Report — Work Completed So Far

Project: Autonomous Multi-Hospital Patient Intake, Scheduling & Pre-Visit Voice Agent / Multi-Tenant Healthcare Discovery, Scheduling, Patient Intake & AI Operations Platform

Current stage: Working prototype with FastAPI backend, Streamlit frontend, mock EHR integration, AI-style conversational appointment agent, scheduling, questionnaire, workflow, notification, reconciliation, audit and operational tracing foundations.

1. Project Goal

The project is being developed as a healthcare platform that can coordinate patient discovery, doctor/availability search, appointment booking, external healthcare-system integration, verification, questionnaire collection and post-booking workflows. The PRD emphasizes that the AI coordinates administrative actions and must not make clinical decisions.

2. Development Environment & Technology

Area	Current implementation
Backend	Python + FastAPI
Database	SQLite + SQLAlchemy ORM
Frontend	Streamlit
External healthcare system	Local mock EHR service using FastAPI + separate SQLite database
AI / conversational layer	Custom Python agent with intent detection, context/session state and capability calls
API communication	HTTP requests between frontend/backend and backend/mock EHR
Version control	Git + GitHub
Repository	Sravani1367/healthcare-ai-platform

3. Backend Components Completed

FastAPI application: Created the main FastAPI application with health endpoint, chat endpoint, database dependency and REST-style endpoints for hospitals, doctors, availability, patients, appointments, questionnaires, workflows, notifications, reconciliation and operations.

SQLAlchemy data layer: Created SQLAlchemy models for Hospital, Doctor, Availability, Patient, Appointment, Questionnaire, QuestionnaireResponse, Workflow, WorkflowExecution, Notification, ReconciliationRecord, AuditEvent and OperationalEvent.

Hospital management: Hospital creation, listing and search functionality has been implemented.

Doctor management: Doctor creation/listing and doctor-related availability handling have been implemented.

Availability: Availability records can be created/listed and available slots can be checked.

Patient management: Patient creation and patient retrieval are available.

Appointment management: Appointment creation, listing, retrieval, cancellation and rescheduling have been implemented.

Questionnaires: Questionnaire creation/listing/retrieval and questionnaire response submission are implemented.

Workflows: Workflow creation, execution/start and execution update operations are implemented.

Notifications: Notification records can be created/stored as part of the workflow.

Reconciliation: Reconciliation records are created when an external appointment operation has an unresolved outcome; records can be retrieved and resolved.

Audit & operations: Audit events, operational events and appointment operation traces are available for observability.

4. Conversational AI / Agent Work Completed

- Implemented a session-based conversational agent.
- Implemented intent detection for cancellation, rescheduling, appointment listing/retrieval, availability checking, booking, doctor search and hospital search.
- Implemented extraction of specialties such as Cardiology, Neurology, Dermatology, Orthopedics and Pediatrics.
- Implemented date extraction for explicit dates and relative dates such as today, tomorrow and day after tomorrow.
- Implemented time extraction for 24-hour and AM/PM formats.
- Implemented doctor discovery and doctor-option disambiguation.
- Implemented collection of missing booking information such as doctor, date and time.
- Implemented real availability checking before booking.
- Implemented patient lookup for the prototype flow.
- Implemented conversational booking, rescheduling, cancellation and appointment lookup/listing flows.
- Implemented session-specific conversation state for the browser/assistant experience.

5. External EHR Integration & Reliability Work

- Created a separate mock EHR service so the project can demonstrate integration without depending on a real hospital system.
- Implemented external appointment creation, verification, search, cancellation and rescheduling.
- Implemented local-to-external appointment ID handling.
- Implemented retry behavior for external appointment creation.
- Implemented an unknown-outcome recovery path by searching the EHR after an uncertain external request.
- Implemented external verification before the local appointment is considered confirmed.
- Implemented reconciliation records for unresolved external failures or verification failures.
- Added EHR failure simulation endpoints so failure/recovery scenarios can be demonstrated.

6. Post-Booking Workflow Foundations

- After successful appointment confirmation, the system can trigger post-booking workflow behavior.
- An active hospital questionnaire can generate a pending questionnaire response for the patient.
- An active appointment-confirmed workflow can be started.
- A patient confirmation notification can be recorded.
- Workflow execution state includes status, current step, error information and timestamps.

7. Streamlit Frontend Completed

- Created a Streamlit-based frontend.
- Added Dashboard, Hospitals, Doctors, Availability, Patients and Appointments pages.
- Added Questionnaires, Workflows and Operations pages.

- Added an AI Assistant page for conversational interaction.
- Added browser session ID handling so the assistant conversation can maintain separate sessions.
- Added appointment booking through available slots.
- Added operational views for reconciliation records, audit events, operation traces and EHR failure simulation.

8. GitHub / Version Control Work Completed

- Initially corrected the Git repository location after Git was accidentally initialized at the parent user directory.
- Initialized Git in the actual project root.
- Added a .gitignore covering virtual environments, Python cache files, SQLite databases and environment files.
- Added backend/requirements.txt.
- Created the first project commit: **b72e076 — Initial healthcare AI platform.**
- Renamed the branch to main.
- Connected the local repository to the GitHub remote.
- Successfully pushed main to GitHub.
- Current local main and origin/main both point to commit b72e076, confirming the current project snapshot is synchronized with GitHub.

9. Current Project File Structure

healthcare-ai-platform/

- backend/
 - agent.py — conversational agent
 - capabilities.py — business/capability layer
 - database.py — SQLAlchemy database configuration
 - ehr_integration.py — mock EHR client integration
 - frontend.py — Streamlit application
 - main.py — FastAPI application and endpoints
 - mock_ehr.py — mock external EHR server
 - models.py — SQLAlchemy models
 - requirements.txt — Python dependencies
 - seed_data.py — prototype seed data

10. What Is Working as a Prototype

The current implementation already demonstrates a substantial administrative healthcare workflow: a patient can interact with the conversational assistant, the system can discover doctors, check availability, select a slot, create an appointment, communicate with the mock EHR, verify the external appointment, synchronize the local state, and trigger post-booking questionnaire/workflow/notification behavior. The operations layer also provides visibility into reconciliation, audit and external-operation events.

11. Important Current Limitations

- Authentication and login are not yet implemented.
- Role-based authorization is not yet implemented.
- Server-side tenant isolation between hospitals is not yet enforced.
- Hospital onboarding/approval lifecycle is not yet implemented as a full workflow.
- Doctor working hours, calendars, leave and blocked periods are not yet modeled comprehensively.
- Appointment concurrency/idempotency needs to be strengthened for production-style double-booking protection.
- Questionnaires currently exist, but a full structured question model and configurable questionnaire associations are still to be built.
- Patient/doctor/hospital dashboards need further development.
- Voice/telephone interface is not yet implemented.
- AI safety boundaries and human escalation need to be formalized.
- Automated unit, integration and end-to-end tests are still required.
- Deployment, architecture documentation, AI documentation and complete project README still need to be prepared.

12. Next Development Phases

Phase	Planned work
Phase 1	Users + authentication + roles
Phase 2	Hospital approval lifecycle + server-side tenant isolation
Phase 3	Calendars + working hours + blocked periods + stronger scheduling rules
Phase 4	Appointment history + idempotency + concurrency protection
Phase 5	Structured questionnaires + conversational questionnaire collection + doctor review
Phase 6	Doctor, patient and hospital dashboards
Phase 7	AI safety controls + human escalation
Phase 8	Automated unit/integration/E2E tests
Phase 9	Voice/telephone prototype
Phase 10	Documentation, deployment and final demo workflow

13. Current Milestone

Milestone reached: Initial working healthcare platform prototype + GitHub baseline completed. The codebase is now backed up in GitHub and ready for incremental implementation of the remaining PRD requirements.